

Digital Image Processing

Homework 4

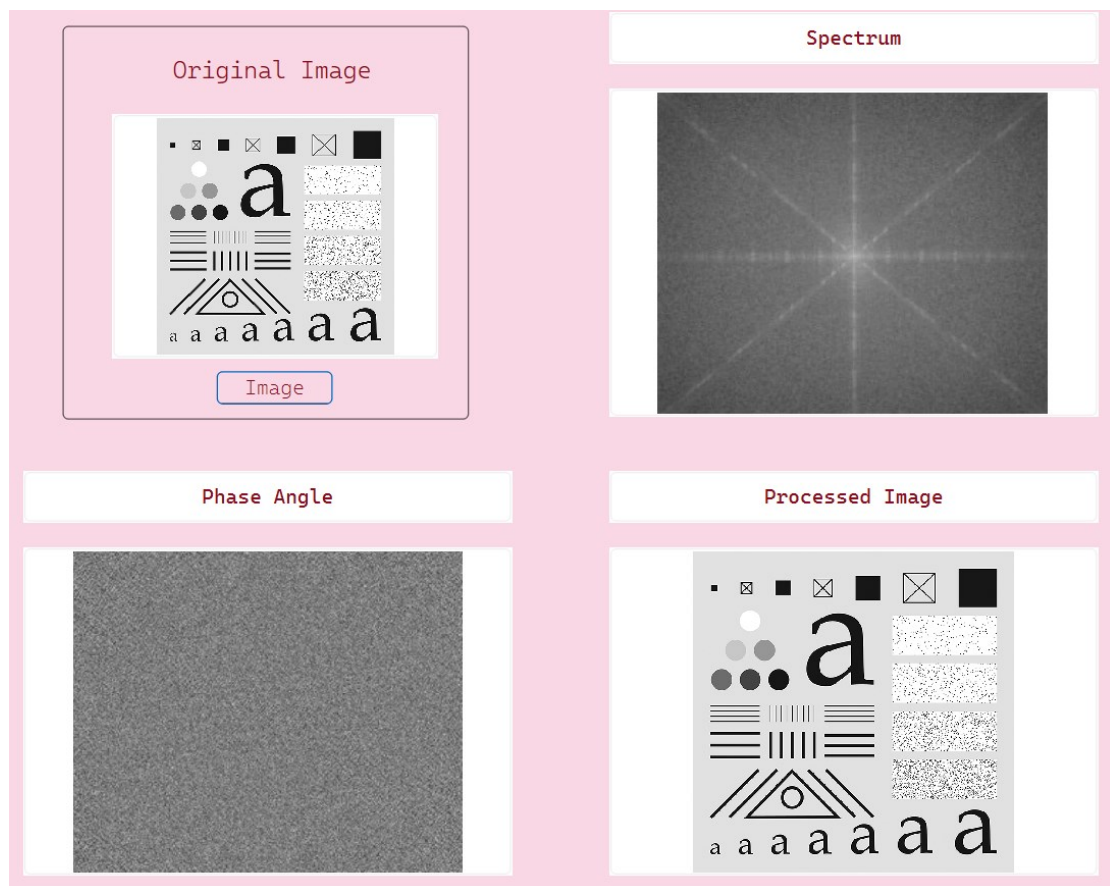
PART1:

Design a program to perform Fourier transform using FFT. Show image spectrum, image phase angle and the IDFT result:

$F_{MIN} = \text{LOG}(1 + \text{ABS}(F_{min}))$ F_{min} is the minimum value of the $F(u,v)$ spectrum

$F_{MAX} = \text{LOG}(1 + \text{ABS}(F_{max}))$ F_{max} is the maximum value of the $F(u,v)$ spectrum

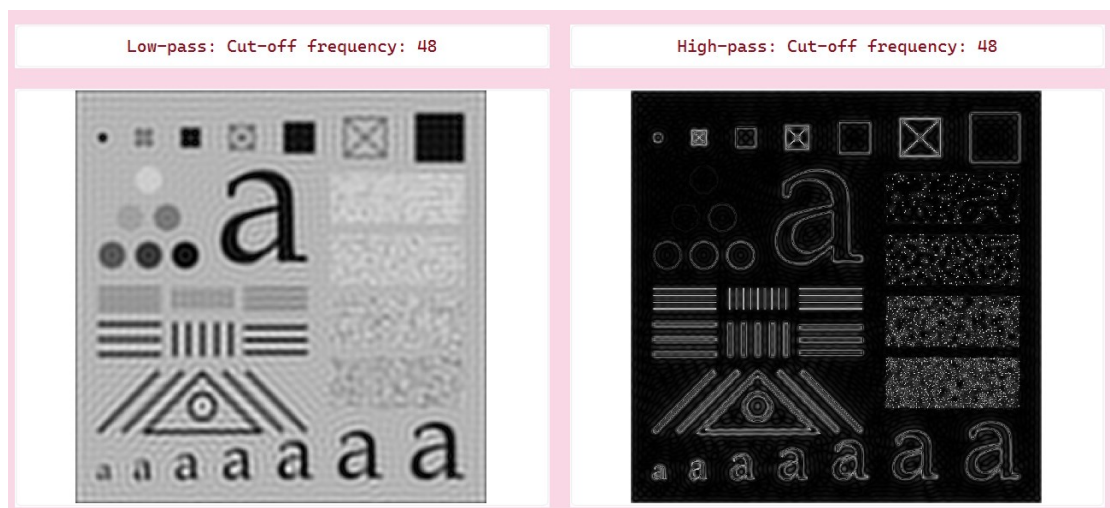
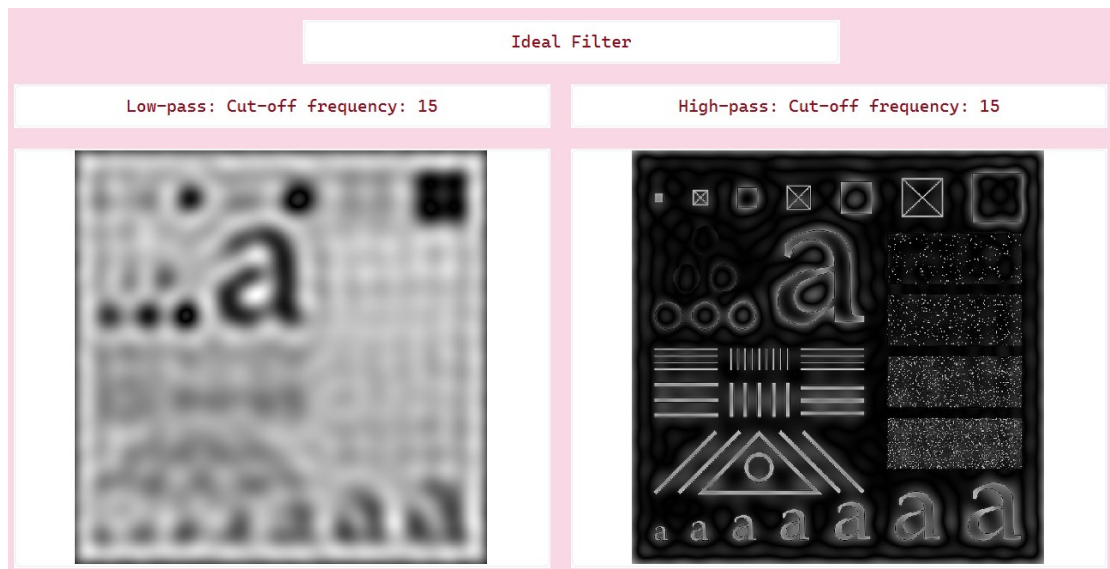
$Y_{NEW}(u,v) = G * [\text{LOG}(1 + \text{ABS}(F(u,v))) - F_{MIN}] / [F_{MAX} - F_{MIN}]$



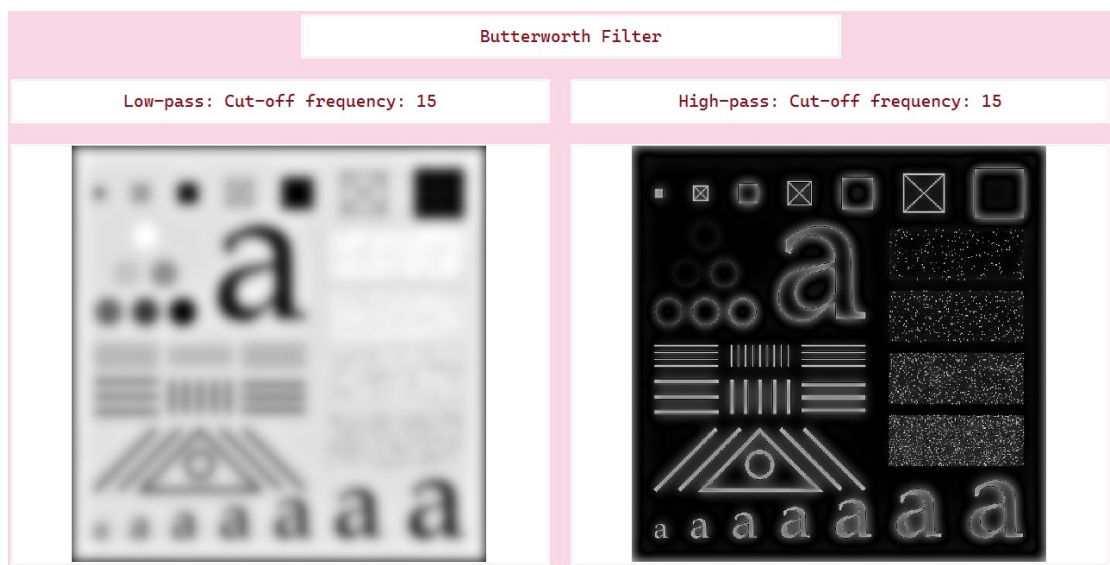
PART2:

Design a program for highpass and lowpass filtering of images using Ideal filter, Butterworth filter, Gaussian filter. Adjust the cut-off frequency on the processed image.

Ideal Filter:

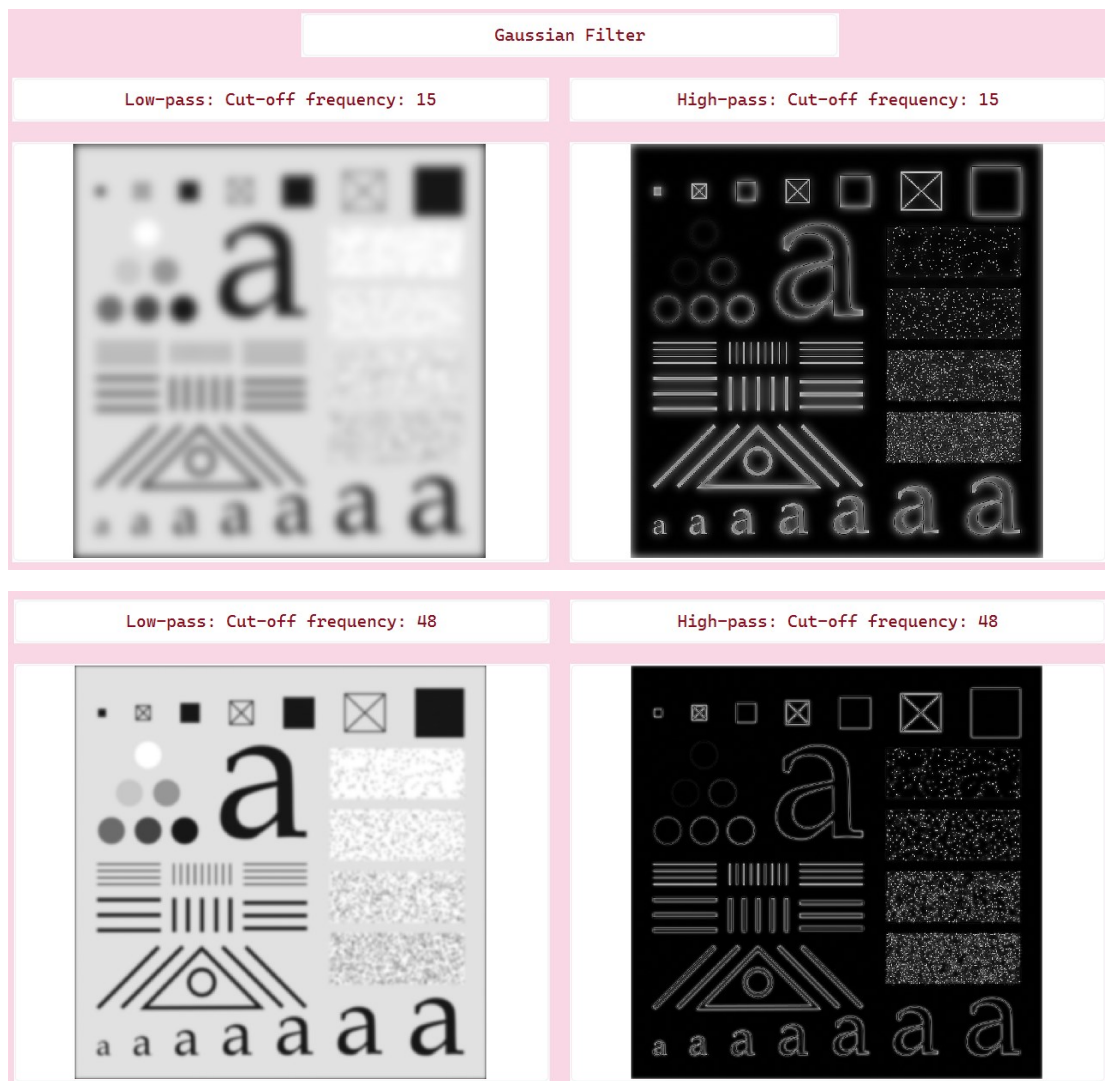


Butterworth Filter:





Gaussian Filter:

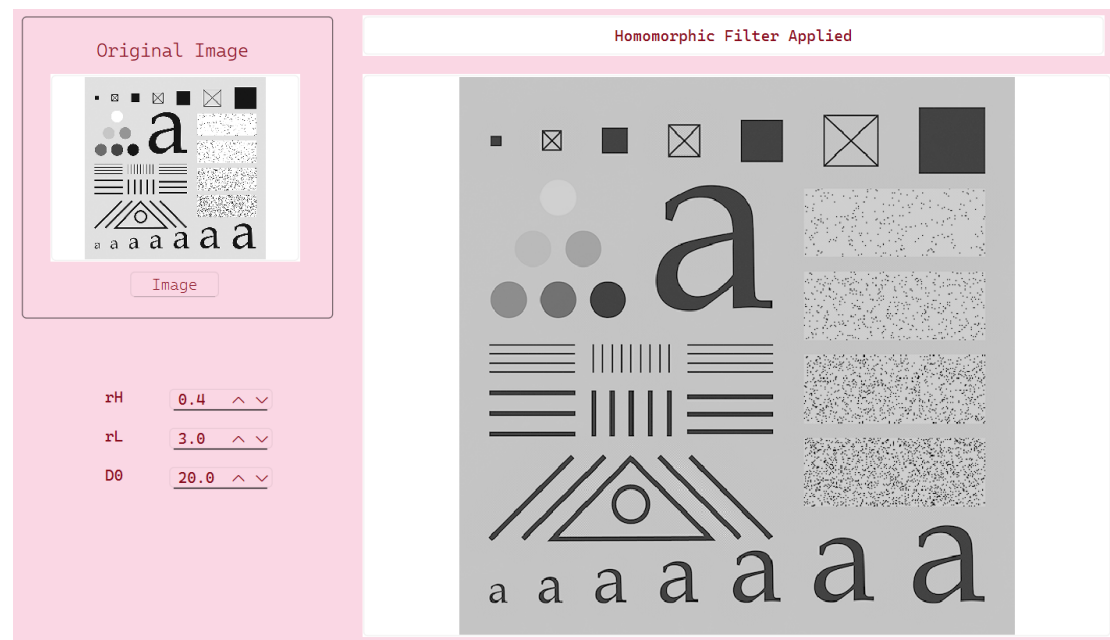


PART3:

Design an image processing program for homomorphic filtering with adjustment of three parameters, γ_H , γ_L , and D_0 .

```
void MainWindow::homomorphic(cv::Mat &filter, float gammaH, float gammaL, float D0, float c){
    int rows = filter.rows;
    int cols = filter.cols;
    int cx = cols / 2;
    int cy = rows / 2;

    for (int i = 0; i < rows; ++i) {
        float *data = filter.ptr<float>(i);
        int y = i - cy;
        for (int j = 0; j < cols; ++j) {
            int x = j - cx;
            float D_uv = std::sqrt(x * x + y * y);
            data[j] = (gammaH - gammaL) * (1 - std::exp(-c * (D_uv * D_uv) / (D0 * D0))) + gammaL;
        }
    }
}
```



PART4:

Design an image processing program to create a motion blurred image then apply a 2-D frequency domain inverse filter and Wiener filter to this blurred image

Inverse Filter:

Motion Blur



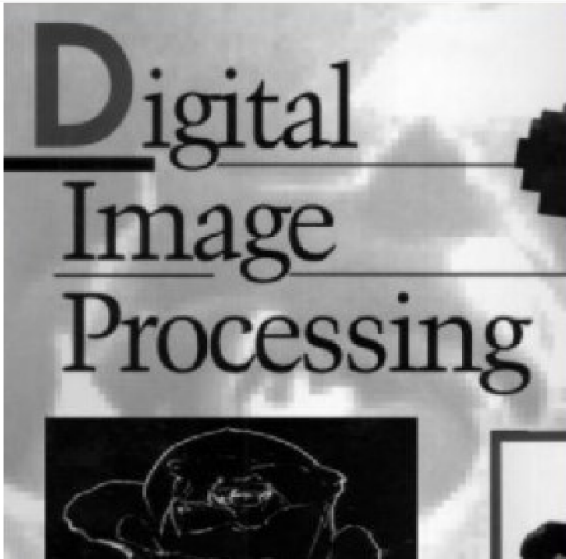
Image

Motion Blur

☒ Inverse Filter

☐ Wiener Filter

Inverse Filter



WeinerFilter:

Motion Blur



Image

Motion Blur

☐ Inverse Filter

☒ Wiener Filter

Weiner Filter

